Fully Dynamic 4-Cycle Counting via Fast Matrix Multiplication: An Exposition

Author A
University of Ljubljana, Faculty of Computer and Information Science
Ljubljana, Slovenia
a@example.si

Author B
University of Ljubljana, Faculty of Computer and Information Science
Ljubljana, Slovenia
b@example.si

Author C
University of Ljubljana, Faculty of Computer and Information Science
Ljubljana, Slovenia
c@example.si

Author D
University of Ljubljana, Faculty of Computer and Information Science
Ljubljana, Slovenia
d@example.si

Abstract

[TODO: write abstract]

CCS Concepts

• **Theory of computation** → **Dynamic graph algorithms**; *Streaming, sublinear and near linear time algorithms*; • **Information systems** → Database query processing.

Keywords

fully dynamic algorithms, subgraph counting, 4-cycles, fast matrix multiplication, incremental view maintenance, cyclic joins

1 Introduction

1.1 Subgraph counting and dynamic graphs

[TODO: motivation paragraph]

1.2 The cyclic-join framing

[TODO: cyclic-join paragraph; insert Figure 1]

1.3 The state of the art

[TODO: ladder discussion + Table 1]

1.4 The Assadi–Shah result

[TODO: state Theorem 1 + the four-bullet surprise discussion]

1.5 What this essay does

[TODO: roadmap paragraph]

2 Preliminaries

2.1 Graphs and subgraph notation

[TODO: notation block]

2.2 Dynamic graph model

[TODO: define fully-dynamic update sequence; worst-case vs amortized]

FIG 1 PLACEHOLDER

Figure 1: A 4-layered graph encoding a cyclic join $A \bowtie B \bowtie C \bowtie D$. One layered 4-cycle highlighted. Adapted from [1], Fig. 1.

Pattern	Upper bound	Lower bound
3-cycle (triangle)	$O(\sqrt{m})$ [4]	$\Omega(m^{1/2-\gamma})$ [3]
4-cycle (this paper)	$O(m^{2/3-\varepsilon})$ [1]	$\Omega(m^{1/2-\gamma})$ [3]
4-clique	$O(m)$ folklore	$\Omega(m^{1-\gamma})$ [2]

Table 1: State of the art for fully dynamic subgraph counting. Lower bounds are conditional (OMv for 3- and 4-cycles; combinatorial 4-clique conjecture for 4-cliques).

2.3 Fast matrix multiplication

[TODO: omega definitions]

2.4 The OMv conjecture

[TODO: OMv paragraph]

3 The cyclic-join equivalence

3.1 Joins as layered graphs

[TODO: joins-as-layered-graphs paragraph; reuse Figure 1]

3.2 Lifting a general graph to a 4-layered graph

[TODO: describe lifting; insert Figure 2]

3.3 Equivalence theorem

Theorem 1 (General–layered equivalence; [1], Claim 8.1). *For the lifted graph G' defined above, the number of length-3 walks from*

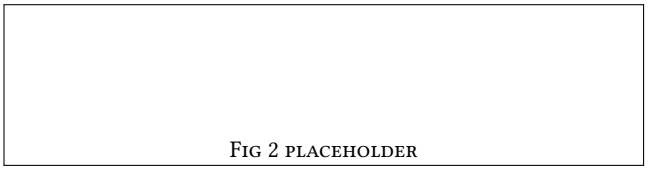

Figure 2: Lifting a general graph G to a 4-layered graph G': every vertex of V is replicated in each of the four layers, and an edge $(u, v) \in E$ becomes four lifted edges between consecutive layers.

Algorithm 1 Maintain 4-cycle count via wedge matrix W.

```
1:  procedure Insert(u, v)
2:      Δ ← Σ_{w∈N(u)} W[w, v]                          ▷ query
3:      C ← C + Δ
4:      for w ∈ N(u) do W[w, v] += 1; W[v, w] += 1
5:      end for
6:      for w ∈ N(v) do W[w, u] += 1; W[u, w] += 1
7:      end for
8:      add (u, v) to E
9:  end procedure
10: procedure Delete(u, v)
11:     remove (u, v) from E
12:     for w ∈ N(u) do W[w, v] −= 1; W[v, w] −= 1
13:     end for
14:     for w ∈ N(v) do W[w, u] −= 1; W[u, w] −= 1
15:     end for
16:     Δ ← Σ_{w∈N(u)} W[w, v]                          ▷ query
17:     C ← C − Δ
18: end procedure
```

$u \in L_1$ to $v \in L_4$ in G' equals the number of length-3 paths from u to v in G.

Proof. **[TODO:** proof body**]** □

3.4 Consequence

[TODO: transition paragraph**]**

4 A simple $O(n)$ algorithm

4.1 The wedge-counting idea

[TODO: wedge identity paragraph**]**

4.2 The algorithm

[TODO: prose around Algorithm 1 explaining the order-of-operations subtlety**]**

4.3 Correctness and complexity

Lemma 2 ([1], Lemma A.1). *Algorithm 1 maintains the exact 4-cycle count of any fully dynamic graph in worst-case update time $O(n)$.*

Proof. **[TODO:** proof body**]** □

4.4 Why $O(n)$ is not enough

Class	Stored products
High in L_1, L_4	$A^{H*} \cdot B_{<i}$; $A^{H*} \cdot B_{<i} \cdot C^{*H}$; $B_{<i} \cdot C^{*H}$
Medium in L_1, L_4	$A^{M*} \cdot B_{<i}$; $B_{<i} \cdot C^{*M}$
Low in L_1, L_4	$A^{L*} \cdot B_{<i,DD}$, $A^{L*} \cdot B_{<i,SS}$, $A^{L*} \cdot B_{<i,SD}$, and three symmetric products with C^{*L}

Table 2: Data structures maintained by the warm-up algorithm. Adapted from [1], Table 1.

[TODO: limitation paragraph**]**

5 The warm-up algorithm

5.1 Setup and assumptions

[TODO: state assumptions; goal $O(m^{2/3-\varepsilon_1})$**]**

5.2 Vertex degree classes

[TODO: define H/M/L on L_1, L_4 and S/D on L_2, L_3 per chunk**]**

5.3 Chunks and lazy maintenance

[TODO: describe chunked maintenance and lazy evaluation**]**

5.4 Data structures

[TODO: walk through the table; gloss the database-language interpretation**]**

Lemma 3 ([1], Lemma 3.2 – statement only). *The worst-case time to update all data structures in Table 2 is $O(m^{2/3-\varepsilon_1})$ per update, with $O(m^{4/3-2\varepsilon_1})$ amortized over chunk transitions.*

5.5 Worked example: the FMM step

Theorem 4 ([1], Claim 3.6). *The matrix product $A^{L*} \cdot B_{i,DD}$ can be computed in time $m^{\omega(2/3+2\varepsilon, 1/3-\varepsilon_1+\varepsilon_2, 1/3-\varepsilon_1+\varepsilon_2)}$.*

Proof. **[TODO:** dimension count + invocation of rectangular FMM**]** □

[TODO: discussion: this constraint (paper Eq (5)) is what *forces* $\varepsilon_1 > 0$, with the system feasible at the numeric values stated in paper §3.4.**]**

5.6 Pulling it together

[TODO: summarize the query case analysis (HH, HM/ML/HL/MM, LL); cite Lemma 3.8**]**

6 The phases idea

6.1 What breaks when A, C are not fixed

[TODO: aggregation breaks once A updates**]**

6.2 Phases vs chunks

[TODO: phase definition + Eq (9)**]**

6.3 Why $\omega < 2.5$ is required

[TODO: the surprise paragraph**]**

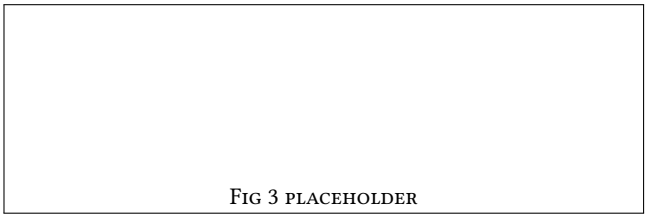

Figure 3: Median per-update time vs. n (log–log) for Simple-Wedge and Naive-Recompute at three densities.

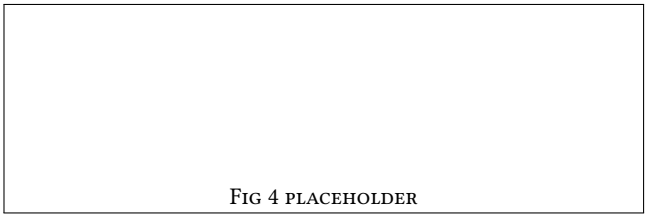

Figure 4: Cumulative time vs. number of updates: crossover where dynamic maintenance overtakes recomputation.

Dataset	n	m	Simple-Wedge $\bar{t}$	Recompute $\bar{t}$
`ca-GrQc`	TBD	TBD	TBD	TBD
`email-Enron`	TBD	TBD	TBD	TBD

Table 3: Real-world graph experiments. $\bar{t}$ in milliseconds, median over 1000 updates.

6.4 Eight path types

[**TODO:** one-paragraph description; do not reproduce Table 2]

6.5 Theorem 1 (paper)

Theorem 5 ([1], Theorem 1 – restated)**.** *There is an algorithm for counting 4-cycles in any fully dynamic graph in worst-case update time $O(m^{2/3-\varepsilon})$, where $\varepsilon > 0$ is a constant depending on ω. With current $\omega = 2.371339$, $\varepsilon \approx 0.0098$; with $\omega = 2$, $\varepsilon = 1/24$.*

[**TODO:** one-line note that the full proof combines §3 warm-up, phases, and the de-assumption arguments in paper §6, §7 – which we do not reproduce]

7 Experiments

7.1 Setup

[**TODO:** implementation details, hardware, datasets (synthetic ER, BA; one SNAP graph)]

7.2 Methodology

[**TODO:** methodology paragraph]

7.3 Results

[**TODO:** discussion of patterns: dense vs sparse, crossover position]

7.4 Connection to the theoretical bounds

[**TODO:** honest caveat + observations connecting back to Section 4]

8 Conclusion

8.1 What we did

[**TODO:** one-paragraph recap]

8.2 Open questions

[**TODO:** open questions paragraph]

References

[1] Sepehr Assadi and Vihan Shah. 2025. An Improved Fully Dynamic Algorithm for Counting 4-Cycles in General Graphs using Fast Matrix Multiplication. arXiv:2504.10748 [cs.DS] PODS 2025, Article 091. https://doi.org/10.1145/3725228.

[2] Kathrin Hanauer, Monika Henzinger, and Qi Cheng Hua. 2022. Fully Dynamic Four-Vertex Subgraph Counting. In *1st Symposium on Algorithmic Foundations of Dynamic Networks (SAND 2022) (LIPIcs, Vol. 221)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 18:1–18:17.

[3] Monika Henzinger, Sebastian Krinninger, Danupon Nanongkai, and Thatchaphol Saranurak. 2015. Unifying and Strengthening Hardness for Dynamic Problems via the Online Matrix-Vector Multiplication Conjecture. In *Proceedings of the Forty-Seventh Annual ACM Symposium on Theory of Computing (STOC)*. 21–30.

[4] Ahmet Kara, Hung Q. Ngo, Milos Nikolic, Dan Olteanu, and Haozhe Zhang. 2020. Maintaining Triangle Queries under Updates. *ACM Transactions on Database Systems* 45, 3 (2020), 11:1–11:46.